Filière : Cyber Sécurité et Intelligence Artificielle (CSIA)

Module : Architecture Logicielle

Rapprot

Nom et Prénom : Abderrazak MOSTAID

Objectif du TP

- Restructurer le monolithe en modules métier distincts
- Définir des boundaries claires entre domaines
- Implémenter des interfaces de communication entre modules
- Appliquer les design patterns (DTO, Mapper, Service Layer)
- Préparer le terrain pour une future migration microservices

Compétences acquises

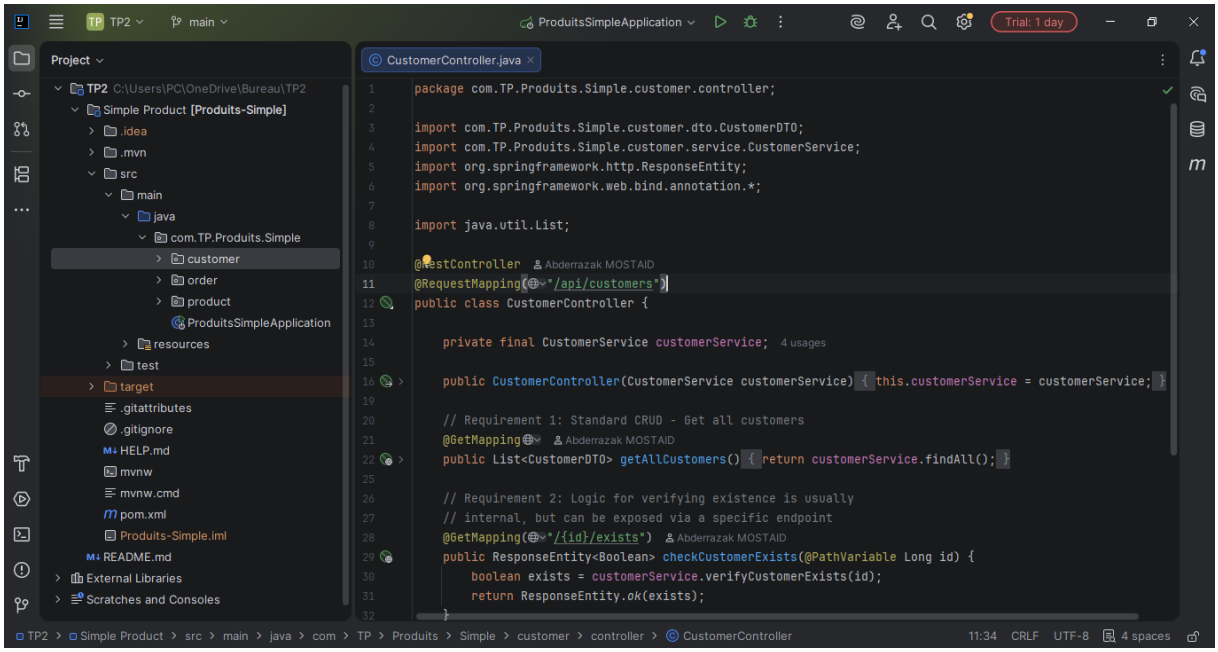
A la fin de ce TP, il faut découvrir comment structurer et appliquer le design patterns (DTO, Mapper, service Layer), aussi préparer une connexion entre les déférents modules pour assurer une meilleure architecture.

Ces compétences à nous permet de découvrir un petit peu des idées concernant l'architecture microservices qui vise à développer chaque service comme une unité indépendant aussi de le déployée et de faire une maintenance à la future.

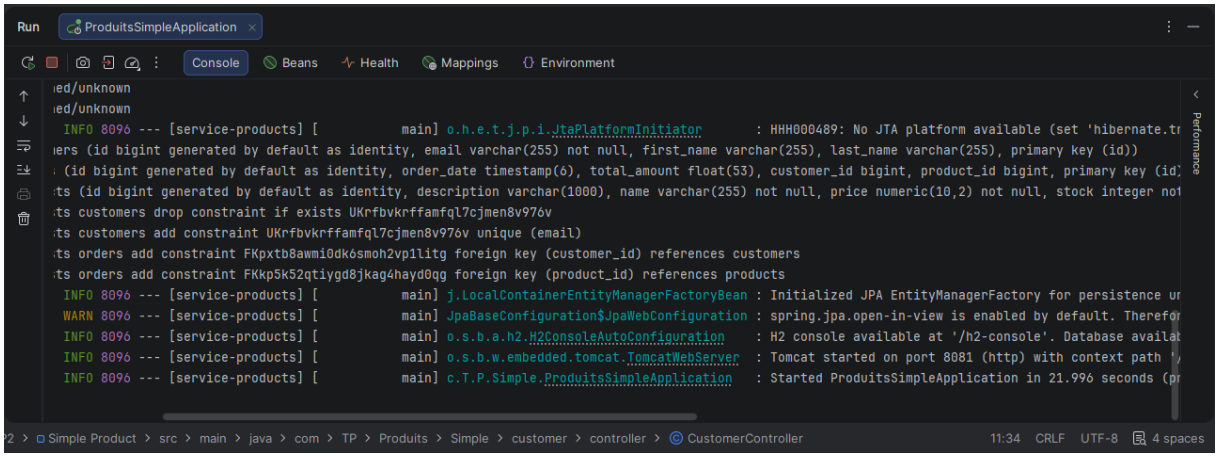
La partie pratique

Pour cette partie, en va se baser sur la premier TP qui est déjà la service Product, et on va essayer de le développer à la fin d'obtenir 3 services dans le même projet, order service comme un module et customer service aussi comme un module.

Structure Global du Projet

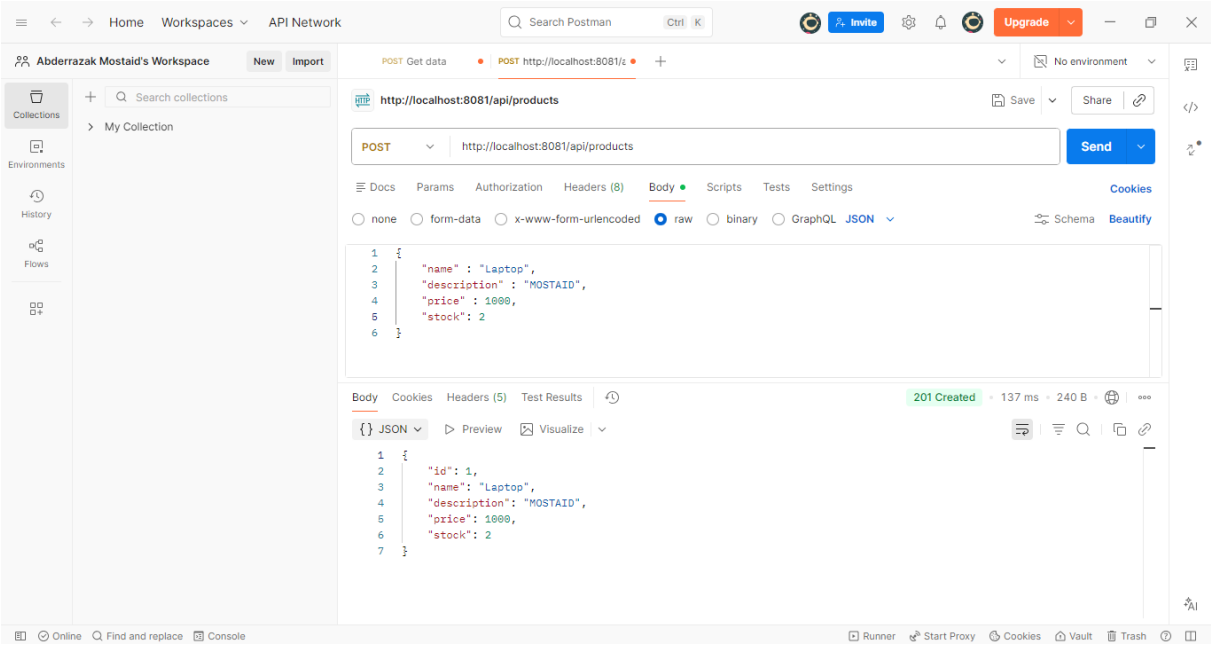


Exécution du code

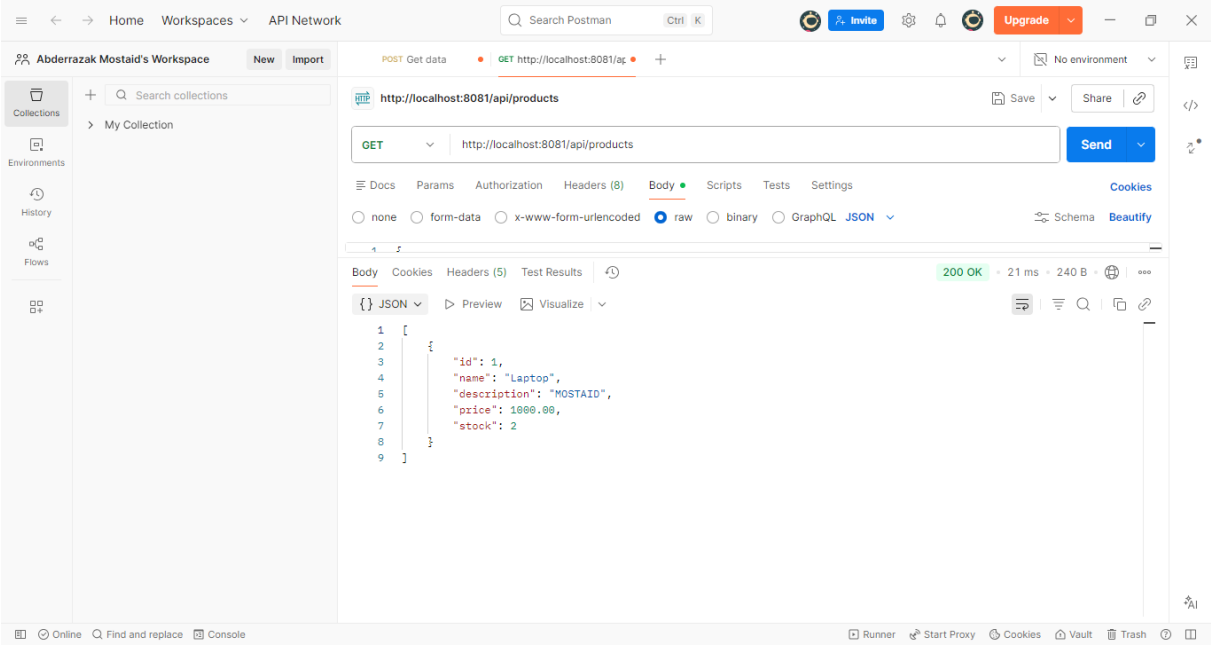


L'exécution du code a été traité avec succès il ne reste que la validation du Endpoint avec des requêtes http via Postman.

Vérification avec Postman (Post request)

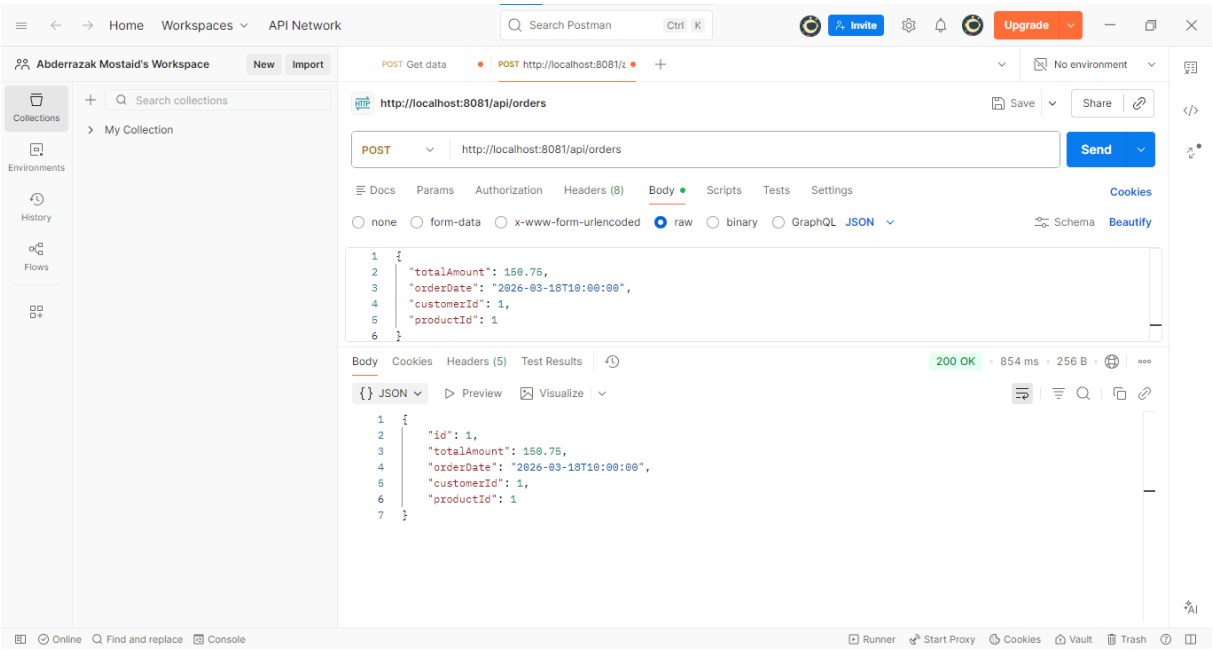


Vérification avec Postman (Get request)

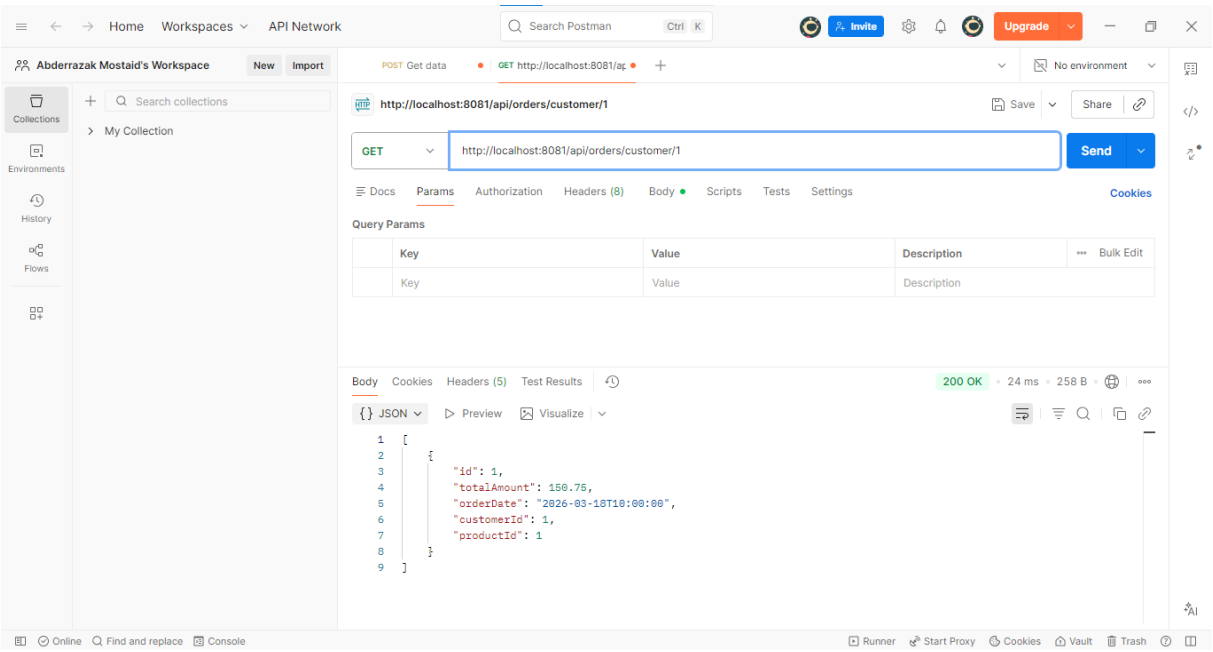


Ici on a vérifié que les requêtes get et post vers le module Product est valide.

Vérification avec Postman (Post)



Vérification avec Postman (Get)



Alors la vérification via Postman a été traité avec succès ce qui montre que la logique du code est valide dans notre cas, et que les modules sont connecté entre eux

Vérification dans le Base de données

```
DROP TABLE
store=# \d
Did not find any relations.
store=# \d
 public | customers        | table    | postgres
 public | customers_id_seq | sequence | postgres
 public | orders           | table    | postgres
 public | orders_id_seq    | sequence | postgres
 public | products         | table    | postgres
 public | products_id_seq  | sequence | postgres


store=# select * from customers;
  1 | Abderrazakmostaid@gmail.com | Abod       | Labooud


store=# select * from orders;
  1 | 2026-03-18 10:00:00 |       150.75 |           1 |          1


store=# select customerID from orders;
ERROR:  column "customerid" does not exist
LINE 1: select customerID from orders;
               ^
HINT:  Perhaps you meant to reference the column "orders.customer_id".
store=# select orders.customer_id from orders;
           1


store=#
```

Alors maintenant lorsque on a fait trois modules dans le même projet on peut même y avoir trois service séparé et c’est exactement le principe de l’architecture microservices.